CORTEX Protocol — Developer Setup Guide

For: The developer completing environment setup and AI agent activation **Date:** April 7, 2026 **Repo:** https://github.com/CortexProtocolBase/Cortex-Protocol **Live Site:** https://www.cortexprotocol.net **Vercel Dashboard:** https://vercel.com/bryces-projects-72528c60/cortex

---

What's Already Done

The CORTEX Protocol is ~90% built and deployed. Here's what's working:

- **5 smart contracts** deployed on Base mainnet (Vault, Token, Staking, Governor, Treasury)
- **Full Next.js frontend** with all pages wired to Supabase-backed API routes
- **AI agent pipeline** — collects market data, analyzes with Claude, validates via guardrails, and now has real DEX execution calldata encoding
- **8 cron jobs** configured on Vercel (snapshot, prices, AI cycle, fees, governance, analytics, indexer, notifications)
- **DEX integration** — Uniswap V3, Aerodrome, Aave V3, Compound V3 adapters with real calldata encoding
- **Oracle** — Chainlink + CoinGecko price feeds
- **Notifications** — Discord, Telegram, Email dispatchers ready
- **Admin dashboard** at `/admin`

What's NOT Working Yet

The AI agent cannot run because it's missing API keys and a wallet with the correct on-chain role. The cron jobs fire but fail silently because the required environment variables aren't set.

YOUR TASKS

Task 1: Get an Anthropic API Key

The AI agent uses Claude (claude-sonnet-4-20250514) to analyze market data and make trade decisions.

1. Go to **https://console.anthropic.com**
2. Create an account or sign in
3. Navigate to **API Keys** in the sidebar
4. Click **Create Key**
5. Copy the key (starts with `sk-ant-...`)
6. Add billing/credits — the AI cycle runs every 10 minutes, each call costs ~$0.01-0.03

Set it on Vercel:

```
# Option A: CLI
npx vercel env add ANTHROPIC_API_KEY production
# Paste the key when prompted

# Option B: Vercel Dashboard
# Go to: https://vercel.com/bryces-projects-72528c60/cortex/settings/environment-variables
# Add: ANTHROPIC_API_KEY = sk-ant-...
# Environment: Production
```

Task 2: Create the AI Agent Wallet

The AI agent needs an Ethereum wallet with the `AI_ROLE` permission on the CortexVault contract. This wallet signs transactions that execute trades through the vault.

Step 1: Create a new wallet

```
# Using cast (Foundry)
cast wallet new

# This outputs:
# Address: 0x...
# Private key: 0x...
```

Or create one in MetaMask and export the private key.

Step 2: Fund it with a tiny amount of ETH on Base

The wallet needs ETH on Base for gas. Send ~~0.005 ETH ($10-15) to the new wallet address on Base network. Gas on Base is cheap (~~$0.001 per transaction).

Step 3: Grant AI_ROLE on the vault contract

The vault uses OpenZeppelin AccessControl. The `AI_ROLE` allows the wallet to call `executeStrategy()`.

```
# The AI_ROLE hash
AI_ROLE=0x9f2df0fed2c77648de5860a4cc508cd0818c85b8b8a1ab4ceeef8d981c8956a6

# Grant the role (must be called by the vault admin/deployer)
cast send 0x3A0799D13c737b341c41004BF9861eBdba28Dcf1 \
  "grantRole(bytes32,address)" \
  $AI_ROLE \
  <NEW_WALLET_ADDRESS> \
  --rpc-url https://mainnet.base.org \
  --private-key <DEPLOYER_PRIVATE_KEY>
```

The `DEPLOYER_PRIVATE_KEY` is already set on Vercel. You can retrieve it:

```
npx vercel env pull .env.local
# Then read DEPLOYER_PRIVATE_KEY from .env.local
```

Step 4: Set the private key on Vercel

```
npx vercel env add AI_AGENT_PRIVATE_KEY production
# Paste the NEW wallet's private key (with 0x prefix)
```

IMPORTANT: Never commit private keys to git. Only set them as Vercel environment variables.

Task 3: Set Up Notifications (Optional but Recommended)

These let the protocol send alerts when trades execute, large deposits happen, or risk thresholds are hit.

Discord Webhook

1. Go to your Discord server
2. Channel Settings → Integrations → Webhooks
3. Click "New Webhook", name it "CORTEX Protocol"
4. Copy the webhook URL
5. Set on Vercel:

```
npx vercel env add DISCORD_WEBHOOK_URL production
```

```
# Paste: https://discord.com/api/webhooks/...
```

Telegram Bot

1. Open Telegram, search for **@BotFather**
2. Send `/newbot`, follow prompts
3. Copy the bot token
4. Add the bot to your group/channel
5. Get the chat ID: `https://api.telegram.org/bot<TOKEN>/getUpdates` (send a message first, then check the JSON for `chat.id`)
6. Set on Vercel:

```
npx vercel env add TELEGRAM_BOT_TOKEN production
npx vercel env add TELEGRAM_CHAT_ID production
```

Task 4: Set Up Sentiment APIs (Optional)

These improve the AI agent's market analysis by reading social sentiment.

Farcaster (via Neynar)

1. Go to **https://neynar.com**
2. Sign up → Create an app
3. Copy API key
4. Set:

```
npx vercel env add NEYNAR_API_KEY production
```

Twitter/X

1. Go to **https://developer.x.com**
2. Create a project and app
3. Generate a Bearer Token (read-only is fine)
4. Set:

```
npx vercel env add TWITTER_BEARER_TOKEN production
```

Task 5: Verify Cron Jobs Are Running

After setting the env vars, redeploy to pick them up:

```
cd /path/to/Cortex-Protocol
npx vercel --prod
```

Then check that crons are firing:

```
# Check Vercel function logs
vercel logs https://www.cortexprotocol.net --since 1h | grep cron
```

Or go to: https://vercel.com/bryces-projects-72528c60/cortex/logs

You should see these crons running:

Cron	Schedule	What It Does
`/api/cron/snapshot`	Every 5 min	Reads vault contract state, stores TVL snapshot
`/api/cron/prices`	Every 1 min	Fetches prices from CoinGecko
`/api/cron/ai-cycle`	Every 10 min	Runs AI analysis + trade execution
`/api/cron/fees`	Hourly	Calculates management + performance fees
`/api/cron/governance`	Every 10 min	Checks proposal expiry
`/api/cron/analytics`	Every 6 hours	Calculates Sharpe, volatility, win rate
`/api/cron/indexer`	Every 5 min	Syncs on-chain deposit/withdraw events
`/api/cron/notifications`	Every 10 min	Sends alerts for large deposits/withdrawals

Task 6: Seed Supabase Data (If Tables Are Empty)

The Supabase tables should be populated by the cron jobs once they're running. But if the site still shows mock data, you may need to run the migrations and seed manually.

Supabase credentials (already on Vercel):

```
npx vercel env pull .env.local
# This downloads: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY
```

Run migrations: Go to https://supabase.com → your project → SQL Editor, and run each file in order:

1. `supabase/migrations/001_create_tables.sql`
2. `supabase/migrations/002_ai_governance_staking.sql`
3. `supabase/migrations/003_indexer_admin.sql`
4. `supabase/migrations/004_rls_policies.sql`

Then run the seed data:

- `supabase/seed.sql`

Architecture Reference

Contract Addresses (Base Mainnet)

Contract	Address
CortexToken (ERC-20)	`0x7A67AFf42d26bDb1A1569C6DE758A4f28e15e4FD`
CortexVault (ERC-4626)	`0x3A0799D13c737b341c41004BF9861eBdba28Dcf1`
Staking	`0x494D4ba8BBe8E9041207A206Cd635af343c9007E`
Governor	`0x11cd3AfcBd99c22B47435DA29E93C26844b8d1Dc`
Treasury	`0xd637A73E056Da2f8761474621B4889c96257d3f3`

AI Agent Trade Flow

```
Cron fires /api/cron/ai-cycle (every 10 min)
  → collector.ts: fetch prices from CoinGecko + on-chain data
```

```
  → analyzer.ts: send to Claude API for analysis
  → Claude returns: HOLD / TRADE / REBALANCE + confidence + trade proposals
  → guardrails.ts: validate tier bounds, rate limits, confidence threshold
  → executor.ts: for each approved trade:
      → adapter.buildSwapCalldata() encodes protocol-specific calldata
      → publicClient.simulateContract() pre-checks the transaction
      → walletClient.writeContract() calls vault.executeStrategy(target, data)
      → Wait for transaction receipt
  → Log full reasoning cycle to ai_reasoning_logs table
```

DEX Adapters

Protocol	Adapter	Actions
Uniswap V3	`src/lib/dex/uniswap.ts`	`exactInputSingle` swaps with auto fee-tier
Aerodrome	`src/lib/dex/aerodrome.ts`	`swapExactTokensForTokens` (volatile + stable)
Aave V3	`src/lib/dex/aave.ts`	`supply()` / `withdraw()`
Compound V3	`src/lib/dex/compound.ts`	`supply()` / `withdraw()` (USDC + WETH comets)

Environment Variables Summary

Variable	Required	Where to Get
`NEXT_PUBLIC_SUPABASE_URL`	Yes	Already set
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Yes	Already set
`SUPABASE_SERVICE_ROLE_KEY`	Yes	Already set
`CRON_SECRET`	Yes	Already set
`DEPLOYER_PRIVATE_KEY`	Yes	Already set
`BASE_RPC_URL`	Yes	Already set
`ADMIN_SECRET`	Yes	Already set
`ANTHROPIC_API_KEY`	**YES — NEED**	https://console.anthropic.com
`AI_AGENT_PRIVATE_KEY`	**YES — NEED**	Create wallet + grant AI_ROLE
`DISCORD_WEBHOOK_URL`	Optional	Discord channel webhook
`TELEGRAM_BOT_TOKEN`	Optional	@BotFather on Telegram
`TELEGRAM_CHAT_ID`	Optional	Telegram API
`NEYNAR_API_KEY`	Optional	https://neynar.com
`TWITTER_BEARER_TOKEN`	Optional	https://developer.x.com

File Structure

```
src/
├── app/                    # Next.js pages and API routes
│   ├── api/v1/            # Public API (vault, trades, AI, governance, staking)
│   ├── api/cron/          # Cron jobs (snapshot, prices, ai-cycle, fees, etc.)
```

```
│   ├── api/admin/         # Admin endpoints (health, stats, treasury, alerts)
│   ├── vault/             # Deposit/withdraw page
│   ├── dashboard/         # Portfolio overview
│   ├── trades/            # Trade history
│   ├── ai/                # AI insights
│   ├── governance/        # Proposals and voting
│   ├── stake/             # Staking
│   ├── strategy/          # Portfolio allocation
│   ├── risk/              # Risk dashboard
│   ├── admin/             # Admin dashboard
│   └── docs/              # API documentation
├── lib/
│   ├── ai/                # AI agent (collector, analyzer, guardrails, executor)
│   ├── dex/               # DEX adapters (Uniswap, Aerodrome, Aave, Compound)
│   ├── oracle/            # Price feeds (Chainlink, CoinGecko, aggregator)
│   ├── sentiment/         # Social sentiment (Farcaster, Twitter, on-chain)
│   ├── risk/              # Risk calculators (Sharpe, VaR, drawdown)
│   ├── notifications/     # Alert dispatchers (Discord, Telegram, Email)
│   ├── indexer/           # On-chain event sync
│   ├── abis/              # Contract ABIs
│   └── supabase.ts        # Database client
├── hooks/                 # Wagmi contract hooks (useVault, useStaking, useGovernance)
├── components/            # React components
└── providers/             # WalletProvider, RealtimeProvider
contracts/
├── src/                   # Solidity contracts
├── test/                  # Foundry tests
└── script/                # Deploy scripts
supabase/
├── migrations/            # SQL schema migrations
└── seed.sql               # Sample data
```

Quick Checklist

- [ ] Get Anthropic API key from https://console.anthropic.com
- [ ] Add `ANTHROPIC_API_KEY` to Vercel env vars
- [ ] Create new Ethereum wallet for AI agent
- [ ] Fund wallet with ~0.005 ETH on Base
- [ ] Grant `AI_ROLE` on vault contract to new wallet
- [ ] Add `AI_AGENT_PRIVATE_KEY` to Vercel env vars
- [ ] Redeploy: `npx vercel --prod`
- [ ] Verify crons running in Vercel logs
- [ ] (Optional) Set up Discord/Telegram notifications
- [ ] (Optional) Set up Neynar/Twitter for sentiment

Once all env vars are set and the AI wallet has the role, the protocol will be fully autonomous — the AI agent will analyze markets every 10 minutes and execute trades through the vault contract.